

Design Scientist 框架说明

本文说明当前本地 `Design-Scientist` 框架的运行逻辑和代码组织。它不是普通的实验流水线，也不是只会维护已有流程的 agent。它的目标是：在有数据基础、目标明确、实验需要多轮迭代的生命科学设计任务中，持续开发和筛选更好的计算设计方法，然后把当前最可信的方法用于下一轮候选设计。

本文分为两部分。第一部分按论文方法部分的方式说明框架如何运行。第二部分从代码层面说明项目结构、文件职责、用户如何开始新任务、哪些节点需要人工接管，以及框架最终会交付什么结果。

一、方法学说明：Design Scientist 如何运行

1. 研究对象和基本假设

Design Scientist 面向的是数据驱动的 wet-lab variant design，而不是一次性训练一个 AI 模型拿 benchmark 分数。典型任务具有几个特征：

1. 已经有若干轮实验数据，数据并不完美，可能存在测量类型差异、系统差异、缺失对照和未完成的模块对比。
2. 目标不是写出一篇传统 AI 论文，而是通过多轮设计-实验-更新，最终得到满足约束的高质量变体。
3. 每一轮新数据不应该让系统从零重新设计策略，而应该更新一个长期研究状态。
4. 算法本身也是研究对象。策略权重、候选分配、uncertainty 使用方式、transfer 假设、lattice repair 机制，都应该被提出、实现、对照、消融和记录。

因此，框架把任务拆成两层：

- **Framework R&D mode**：开发新的计算设计方法。它关注方法是否新颖、是否可执行、是否击败 baseline、是否能解释失败。
- **Project execution mode**：用当前已经通过验证的方法，为一个具体生物设计项目生成候选池、推荐面板和人工审查包。

这两层共享同一个核心思想：所有结论都必须落在 artifact 上，而不是留在对话历史里。

2. 总体运行架构

Design Scientist 的运行可以看作一个本地 Research OS。它由五层组成：

1. Reference layer

保留 `AI-Scientist`、`AI-Scientist-v2` 和 `DeepScientist` 源码作为只读参考。框架不会从这些仓库直接 import 生产代码，而是通过 `audit-references` 把可借鉴的结构写入 `framework/reference_audit.md` 和 `framework/reference_components.json`。目前借鉴点包括：`AI-Scientist v1` 的 `idea/experiment/writeup/review` 产物链，`AI-Scientist v2` 的 `stage manager`、`tree search` 和 `journal`，`DeepScientist` 的 `quest`、`findings memory`、`failure memory`、`BO` 式 `exploration/exploitation` 和 `human takeover`。

2. Research OS layer

每个长期研究任务被初始化成一个 `quest`。`quest` 不是一次 run，而是一个可以持续追加发现和失败路径的研究空间。核心文件是 `framework/quest.yaml`、`framework/research_map.json`、`framework/findings_memory.jsonl` 和 `framework/failure_memory.jsonl`。

3. Literature and method layer

文献不是装饰，而是方法发明的输入。框架先规划查询，再通过 PubMed、bioRxiv、arXiv 和可选 Semantic

Scholar 获取 paper cards, 随后抽取 method modules、baseline、evaluation protocol、failure modes 和 reusable ideas, 生成 `algorithm_spec.md` 和 `method_registry.yaml`。

4. Scientist manager layer

参考 AI-Scientist v2, 框架把一次科学家循环拆成阶段: baseline reproduction、literature grounded ideation、policy implementation、debug and repair、creative research、ablation and stress、selection and report。每个阶段写入 journal、stage progress 和 route tree。

5. Design kernel layer

这是本框架和 AI-Scientist 最大的区别。这里的 experiment 不是训练模型, 而是提出一个 candidate selection policy, 运行多轮 synthetic wet-lab replay, 对比 baseline 和 ablation, 并验证它是否产生不被数据支持的 claim。

3. 形式化问题定义

在第 t 轮, 系统持有一个已观察集合:

$$O_t = \{\text{历史实验观察、端点、测量类型、来源、证据层级、轮次}\}$$

同时有一个可设计候选集合:

$$C_t = \{\text{满足基本约束的候选变体、模块组合、对照、重复、lattice repair 设计}\}$$

一个策略 π 接收 O_t 和 C_t , 输出预算为 B 的下一批实验:

```
select_batch(observed, candidates, budget, round_index, rng) -> list[str]
```

这就是当前统一的 policy API。无论是内置 baseline、mechanism-aware 默认策略, 还是 Codex 生成的新方法, 都必须走同一个 API。这样做的目的很直接: 所有方法在相同输入、相同预算、相同模拟世界和相同指标下比较, LLM 不能用文字解释替代实际选择结果。

4. 证据和 claim 的处理方式

框架把 evidence 分成几个层级:

- `primary_matched`: 精确匹配的主要实验对比。
- `secondary_near_matched`: 近似匹配, 但仍需标注限制。
- `descriptive`: 描述性事实, 例如数据库存、系统角色、缺失边。
- `model_derived`: 模型或模拟生成的信息。

这个区分对生命科学设计很关键。比如某个模块在一个系统里表现好, 不代表它在另一个 target system 中具备因果作用。框架要求把“模块学习数据”和“目标系统设计数据”分开, 并且在缺少 matched edge 时禁止产生模块因果结论。

在代码上, `table_io.py` 和 `validators.py` 强制保护 identifier。典型例子是 1E62: CSV 读取时不能被 pandas 自动读成科学计数法。框架会把已知错误形式如 `1e+62` 规范化为 `1E62`, 并在 validation 中拒绝 artifact 里残留的 `1e+62`。

5. 文献检索 v2 流程

文献检索不是单次关键词搜索, 而是 staged workflow:

1. Query planning

读取 `framework/literature_queries.yaml`。如果用户没有写查询，则从 domain 生成保守 query。

2. Source search

调用 PubMed、bioRxiv、arXiv 和 Semantic Scholar adapter。Semantic Scholar 只有在 `S2_API_KEY` 存在时 live 启用；没有 key 时记录 skip，不让主流程崩溃。所有 adapter 都有 offline fixture mode，测试不依赖真实网络。

3. Raw cache

原始响应写到 `framework/cache/literature_raw/`，避免结果来源不可追踪。

4. Normalization

每篇文献标准化成 paper card，包含 source、query_id、external_id、title、summary、year/date 和 provenance。

5. Ranking and deduplication

按 source priority、citation count、recency 和 method keyword score 做 deterministic ranking，同时跨来源去重。

6. Trace and graph

写出 `literature_search_trace.json`、`paper_scores.csv` 和 `citation_graph.json`。后续每个方法假设都应该能追溯到某些 paper cards。

6. 从文献到方法模块

`extract-methods` 读取 `framework/paper_cards.json`，把文献归纳成可复用的 method modules。当前内置的抽取模板包括：

- batch active learning for variant design
- constrained Bayesian optimization
- adaptive decision-value allocation
- matched contrast lattice
- mechanism-aware antibody design

每个 module 会记录 problem setting、data regime、algorithm family、acquisition mechanism、baselines、evaluation protocol、failure modes 和 reusable ideas。抽取结果写入：

- `framework/method_modules.json`
- `framework/literature_map.md`
- `framework/research_gap_matrix.csv`
- `framework/algorithm_spec.md`
- `framework/method_registry.yaml`
- `framework/method_hypotheses.md`

如果没有可用 paper cards，框架不会伪造方法，而是写 `framework/method_extraction_failure.md` 并返回 failure。这一点很重要：没有文献依据时，系统宁可失败，也不能凭空生成看似合理的算法。

7. 方法发明：controlled method nodes

方法发明不是让 LLM 直接改核心框架。框架把每个候选方法放进隔离 node workspace：

```
runs/<run_id>/nodes/<node_id>/
```

每个 method node 必须产出固定文件：

- `proposal.json`
- `method.py`
- `candidate_policy.json`
- `novelty_report.json`
- `benchmark_metrics.json`
- `validation_report.json`
- 兼容旧接口的 `manifest.json`

其中 `method.py` 必须提供可调用的 `select_batch`。`proposal.json` 必须写明 method hypothesis、literature basis、algorithm mechanism、expected advantage、failure modes 和 planned ablations。

Codex 可在 node workspace 内生成代码，但不能直接修改主框架。`method_nodes.py` 在执行前后做文件系统 snapshot，并要求所有写入都留在 node workspace 内。越界写入、缺 artifact、malformed JSON、runtime exception、validation_report 非 valid，都会把 node 标记为 invalid。

8. 新颖性和 baseline clone guard

框架不允许一个所谓“新方法”只是 baseline 的换名包装。每个 generated policy 在 replay 中会计算：

- `selection_overlap_random_feasible`
- `selection_overlap_fixed_mix`
- 与 source baseline 的 selection overlap
- `novelty_score`

如果某个 node 与 baseline 的选择完全重合，会被标记为 behavioral baseline clone。clone method 不能成为 selected method。`novelty_report.json` 先由 node 写出初始声明，再由 synthetic replay 的真实测量结果刷新，避免 node 自己虚构新颖性。

9. Synthetic multi-world wet-lab replay

Benchmark 不是单世界模拟，而是 multi-world suite：

- `additive`：模块效应近似可加。
- `epistatic`：存在 pair/triple interaction。
- `confounded_transfer`：来源系统和目标系统之间存在 transfer confounding。
- `noisy_endpoint`：端点测量噪声更强。
- `sparse_early_round`：早期观察稀疏。

每个世界都有完整的 latent design lattice。策略只能看到已经揭示的观测，必须按轮次花预算选择新候选。系统随后用隐藏真值评估策略表现。

核心指标包括：

- best feasible utility
- hit rate
- regret proxy
- false claim rate
- evidence coverage

- round efficiency
- novelty score
- baseline overlap

一个 selected method 必须在多数世界中击败 `random_feasible` 和 `fixed_mix`，false claim rate 不能变差，并且 novelty 不能退化为 baseline clone。如果没有方法通过，run 仍会生成完整报告，但 `review-framework` 返回 error。

10. Deterministic selection

LLM 在框架里的职责是生成、解释和批判。最终裁判不是 LLM，而是 deterministic benchmark。

排序分数大致由以下因素组成：

```
ranking_score
= best feasible utility
+ hit rate contribution
+ evidence coverage contribution
+ round efficiency contribution
- regret penalty
- false claim rate penalty
```

这意味着框架并不盲目追求可解释性。如果 interpretability 牺牲太多 final design performance，它不会被选中。更准确地说，框架追求的是 performance-first 的可解释性：解释性必须帮助减少错误 claim、提升 transfer 判断或改善下一轮实验设计，而不能成为替代结果的理由。

11. Journal、memory 和 report

一次 scientist run 会写出：

- `runs/<run_id>/scientist_journal.json`
- `runs/<run_id>/stage_progress.json`
- `runs/<run_id>/route_tree.json`
- `runs/<run_id>/benchmark_results.csv`
- `runs/<run_id>/benchmark_summary.csv`
- `runs/<run_id>/ablation_results.csv`
- `runs/<run_id>/method_report.md`

同时，Research OS 会向：

- `framework/findings_memory.jsonl`
- `framework/failure_memory.jsonl`

追加可复用发现和失败路径。失败路径被保留下来，是为了避免下一轮重复踩同一个坑。

12. Human review 的位置

框架可以自动运行很多步骤，但它不是无人值守湿实验决策器。人工接管点包括：

1. 定义 **quest** 时：确认 domain、objective、可用数据和约束是否准确。
2. 文献检索后：检查 paper cards 是否偏题，是否缺关键方法方向。

- 3. **方法抽取后**：判断 method modules 是否合理，是否需要手工加入领域方法。
- 4. **Codex node 生成后**：检查 proposal 是否真正提出了新机制，而不是包装 baseline。
- 5. **benchmark 后**：检查 selected method 是否只是 synthetic world 过拟合。
- 6. **validation 出 error 时**：必须人工决定修复数据、修复代码、降低 claim，还是停止推进。
- 7. **进入真实 wet-lab 前**：任何 panel recommendation 都必须通过 human review packet，不应直接当作最终实验单。

二、代码层面说明：项目如何组织和使用

1. 总目录

主工程位于：

```
/Users/ziyang/Research/Design-Scientist
```

主要目录如下：

路径	作用
src/design_scientist/	Python package 主体，包含 CLI、schema、文献、方法、benchmark、validation、report、project loop。
tests/	单元测试和集成测试，覆盖 adapter、method extraction、synthetic replay、node guard、validation、CLI。
references/AI-Scientist/	AI-Scientist v1 只读参考。
references/AI-Scientist-v2/	AI-Scientist v2 只读参考。
references/DeepScientist/	DeepScientist 只读参考。
projects/anti_hbsag/	当前保留的示例项目目录。用户这次要求不推进 HBsAg，只保留框架和 identifier 修复。
output/pdf/	本说明文档的 Markdown 和 PDF 输出目录。
pyproject.toml	包定义、依赖、CLI entry point。
uv.lock	uv 锁文件。
README.md	工程入口说明和基础命令。

2. CLI 入口

CLI 定义在 `src/design_scientist/cli.py`。入口命令由 `pyproject.toml` 注册：

```
[project.scripts]
design-scientist = "design_scientist.cli:main"
```

主要命令分为两组。

Framework R&D 命令：

```
design-scientist init-framework
design-scientist audit-references
design-scientist literature-search
design-scientist extract-methods
design-scientist develop-method
design-scientist benchmark-methods
design-scientist run-scientist
design-scientist review-framework
```

Project execution 命令：

```
design-scientist init
design-scientist build-state
design-scientist literature
design-scientist hypotheses
design-scientist retrospective
design-scientist merge-data
design-scientist run-dry-panel
design-scientist run-full
design-scientist review
design-scientist codex-task
```

3. Source 文件职责

下面按文件说明 `src/design_scientist/` 的组织逻辑。

文件	职责
<code>__init__.py</code>	package 标识文件。
<code>cli.py</code>	命令行入口，解析参数并分发到 framework R&D 或 project execution 模块。
<code>schemas.py</code>	核心 dataclass 定义，包括 <code>ProjectSpec</code> 、 <code>DataContract</code> 、 <code>DesignState</code> 、 <code>Candidate</code> 、 <code>FrameworkSpec</code> 、 <code>PaperCard</code> 、 <code>MethodModule</code> 、 <code>ScientistJournal</code> 、backend request/result 等。
<code>artifacts.py</code>	集中定义项目和框架 artifact 名称，提供缺失 artifact 检查。
<code>io.py</code>	YAML/JSON 读写、目录创建、路径辅助函数。
<code>table_io.py</code>	identifier-safe CSV reader。保证 <code>system</code> 、 <code>variant_id</code> 、 <code>module_id</code> 、 <code>paper_id</code> 等列按字符串处理，并修复/检测 1E62 科学计数法问题。
<code>framework.py</code>	<code>init-framework</code> 的实现，创建 <code>framework_spec.yaml</code> 、 <code>literature_queries.yaml</code> 、 <code>benchmark/run</code> 目录，并初始化 Research OS。
<code>research_os.py</code>	quest、research map、findings memory、failure memory 的创建和 JSONL 追加。
<code>reference_audit.py</code>	对 AI-Scientist v1/v2 和 DeepScientist 的本地参考仓库做 deterministic audit，写 reference audit artifacts。
<code>literature_sources.py</code>	PubMed、bioRxiv、arXiv、Semantic Scholar adapter，支持 live search、raw cache 和 offline fixtures。

文件	职责
<code>literature_pipeline.py</code>	Literature Search v2 主流程：query planning、source search、ranking、dedupe、trace、citation graph、paper cards。
<code>literature.py</code>	早期项目执行路径中的 seed paper cards 和 gap matrix 工具，主要保留兼容旧流程。
<code>method_extraction.py</code>	从 paper cards 抽取 method modules、literature map、gap matrix、algorithm spec、method registry 和 method hypotheses。
<code>methods.py</code>	早期方法假设生成工具，主要服务 project execution 的旧入口。
<code>policies.py</code>	统一 policy API 和 baseline/default policies，包括 <code>random_feasible</code> 、 <code>top_observed</code> 、 <code>greedy_utility</code> 、 <code>fixed_mix</code> 、 <code>pure_uncertainty</code> 、 <code>pure_lattice_repair</code> 、 <code>mechanism_aware</code> 。
<code>synthetic_replay.py</code>	多世界 synthetic wet-lab replay benchmark。定义 latent module effects、interaction、noise、multi-objective utility、metrics、summary 和 ablation 输出。
<code>method_nodes.py</code>	method node contract、加载、执行、artifact validation、path guard、baseline clone 初步检查。
<code>scientist_search.py</code>	scientist loop 编排。负责 literature/extraction gate、node 生成、Codex backend 调用、node execution、benchmark ranking、journal、stage progress、route tree 和 memory 追加。
<code>framework_validation.py</code>	<code>review-framework</code> 的实现，检查 framework spec、literature artifacts、method artifacts、benchmark、journal、selected node、report、majority win、novelty 和 unsupported claims。
<code>method_report.py</code>	从 journal、literature、benchmark、node artifacts 生成 <code>method_report.md</code> 。
<code>backends/base.py</code>	定义 <code>ModelBackend</code> 和 <code>WorkspaceAgentBackend</code> protocol。
<code>backends/codex_cli.py</code>	本地 Codex CLI backend，通过 <code>codex exec</code> 执行 read-only 或 workspace-write 任务，支持 <code>--output-schema</code> 。
<code>projects/anti_hbsag.py</code>	anti-HBsAg 示例项目初始化，创建 project/data contract/estimands/standardized/state 目录。
<code>state.py</code>	从 allowlisted data contract 构建 project design state 和 evidence cards，并统一使用 identifier-safe IO。
<code>evidence.py</code>	构建当前 anti-HBsAg 示例中的 evidence cards，强调 sdAb 和 1E62 角色分离。
<code>candidates.py</code>	Project execution 的候选生成 operators，例如 add module、remove module、complete missing edge、complete square、coverage validation、repeat/control。
<code>acquisition.py</code>	候选和 panel 的机制感知打分函数。
<code>panel.py</code>	project execution 的 panel selection 和 baseline comparison。
<code>search.py</code>	旧版 controlled search，用于 dry-panel 项目执行路径。
<code>pipeline.py</code>	高层 project execution pipeline，包括 <code>run_dry_panel</code> 和 <code>run_full_workflow</code> 。
<code>reporting.py</code>	写 human review packet。
<code>journal.py</code>	旧版 design journal 读写。
<code>retrospective.py</code>	project execution 的 retrospective policy evaluation。

文件	职责
hypotheses.py	project execution 的 hypothesis board 构建。
project_history.py	项目历史事件 JSON 追加。
data_merge.py	新数据 intake, 先进入 pending schema audit, 而不是直接进入 evidence。
validators.py	project execution validation, 检查 data contract、state、run artifacts、baseline comparison、unsupported claims 和 identifier 问题。

4. 测试组织

tests/ 按模块对应源码:

测试文件	主要覆盖
test_framework_cli.py 、 test_framework_integration.py	framework CLI 和端到端离线流程。
test_reference_audit.py	reference audit artifacts。
test_research_os.py	quest、research map、findings/failure memory。
test_literature_adapters.py 、 test_literature_methods.py 、 test_method_extraction.py	文献 adapter、paper card、method extraction。
test_policy_api.py 、 test_synthetic_replay.py	policy API、多世界 replay、baseline metrics、majority gates。
test_method_nodes.py 、 test_scientist_search.py	method node contract、path guard、fake Codex、scientist journal、baseline clone rejection。
test_framework_validation.py	review-framework 的 artifact 和 selected method 验证。
test_table_io.py 、 test_state_build.py 、 test_search_and_validation.py	identifier-safe IO、1E62 修复、project state 和 validation。
test_project_init.py 、 test_full_workflow.py 、 test_panel_selection.py 、 test_retrospective.py 、 test_hypotheses.py 、 test_data_merge.py 、 test_codex_backend.py	旧项目执行路径、数据合并、Codex backend 和 panel 相关行为。

测试 fixtures 位于 tests/fixtures/literature/ , 用于 PubMed、bioRxiv、arXiv 和 Semantic Scholar 的离线文献搜索测试。

5. 用户如何开始一个新 framework R&D 任务

建议从一个新的目录开始, 不直接在 projects/anti_hbsag 上实验框架本身:

```
cd /Users/ziyang/Research/Design-Scientist

uv run design-scientist init-framework /tmp/my_design_scientist_project \
  --domain protein_variant_design
```

```
uv run design-scientist audit-references /tmp/my_design_scientist_project
```

如果只是测试本地流程，先用 offline fixtures：

```
uv run design-scientist run-scientist /tmp/my_design_scientist_project \  
  --offline-fixtures \  
  --max-papers 60 \  
  --nodes 4 \  
  --rounds 3
```

```
uv run design-scientist review-framework /tmp/my_design_scientist_project
```

如果要做真实文献检索，可以设置：

```
export S2_API_KEY=...  
export NCBI_EMAIL=...  
export NCBI_TOOL=design_scientist  
export NCBI_API_KEY=... # 可选
```

然后运行：

```
uv run design-scientist literature-search /tmp/my_design_scientist_project --max-  
papers 60  
uv run design-scientist extract-methods /tmp/my_design_scientist_project  
uv run design-scientist develop-method /tmp/my_design_scientist_project --nodes 4  
uv run design-scientist benchmark-methods /tmp/my_design_scientist_project --rounds 3  
uv run design-scientist review-framework /tmp/my_design_scientist_project
```

如果要让本地 Codex 参与生成 method node：

```
uv run design-scientist run-scientist /tmp/my_design_scientist_project \  
  --max-papers 60 \  
  --nodes 4 \  
  --rounds 3 \  
  --use-codex
```

Codex 生成只允许写 node workspace，不允许改主框架。

6. 用户如何开始一个新的具体项目

具体项目执行不是框架 R&D。它的目标是用当前方法服务一个实际数据项目。

基本流程是：

```
uv run design-scientist init /path/to/project --project-id anti_hbsag  
uv run design-scientist merge-data /path/to/project /path/to/new_data --round-label  
round_1  
uv run design-scientist build-state /path/to/project
```

```
uv run design-scientist run-dry-panel /path/to/project --budget 24
uv run design-scientist review /path/to/project
```

`merge-data` 不会把新数据直接当作证据使用，而是先放入 `pending schema audit`。只有通过 `data contract` 和 `schema audit` 的标准化表，才应该进入 `build-state`。

7. 任务过程中需要人工接管的时刻

从用户角度，建议在这些时刻停下来检查：

1. `init-framework` 后

检查 `framework/quest.yaml` 和 `framework/literature_queries.yaml`。如果 `domain` 写错，后续文献和方法都会偏。

2. `literature-search` 后

检查 `paper_cards.json`、`paper_scores.csv`、`literature_search_trace.json`。确认文献是否真的围绕方法，而不是只围绕具体生物对象。

3. `extract-methods` 后

检查 `literature_map.md`、`research_gap_matrix.csv`、`algorithm_spec.md`。如果文献抽取太弱，需要人工补充 `paper cards` 或修改 `query`。

4. `develop-method` 后

检查每个 `node` 的 `proposal.json` 和 `method.py`。重点看它是否提出了新的 `acquisition/allocation/uncertainty/transfer` 机制，而不只是 `baseline` 换皮。

5. `benchmark-methods` 或 `run-scientist` 后

检查 `benchmark_results.csv`、`benchmark_summary.csv`、`ablation_results.csv`。如果 `selected method` 只在一个 `synthetic world` 赢，不能直接信任。

6. `review-framework` 报 error 时

必须人工处理。`error` 代表缺关键 `artifact`、`selected node` 无效、`baseline clone`、`majority win` 不达标、`identifier` 不可信或 `report` 不完整。

7. 真实 `wet-lab` 前

即使 `dry-run panel` 通过 `validation`，也需要看 `human_review_packet.md`。框架输出的是候选决策证据，不是自动下实验单。

8. 框架最终会给出什么结果

Framework R&D run 的最终结果是一套可审查的方法证据包：

- `scientist_journal.json`：完整记录文献快照、`node`、`benchmark`、`selected node` 和失败路径。
- `stage_progress.json`：每个 `stage` 的状态和对应 `artifact`。
- `route_tree.json`：方法搜索树和 `selected branch`。
- `benchmark_results.csv`：每个方法在每个 `synthetic world` 上的指标。
- `benchmark_summary.csv`：跨世界聚合结果和 `majority win`。
- `ablation_results.csv`：方法组件消融。
- `nodes/<node_id>/proposal.json`：方法假设和文献依据。
- `nodes/<node_id>/method.py`：可执行 `policy`。
- `nodes/<node_id>/novelty_report.json`：新颖性和 `baseline overlap`。
- `nodes/<node_id>/benchmark_metrics.json`：节点级 `benchmark` 结果。
- `method_report.md`：面向人的方法报告。

- `framework/findings_memory.jsonl` : 可复用发现。
- `framework/failure_memory.jsonl` : 失败路线。

Project execution run 的最终结果是一套候选面板证据包：

- `candidate_pool.csv` : 候选池。
- `panel_recommendation.csv` : 推荐面板。
- `policy_comparison.csv` : baseline 对照。
- `policy_metrics.json` : 策略指标和 unsupported claims。
- `validation_report.json` : 验证结果。
- `decision_report.md` : 决策摘要。
- `human_review_packet.md` : 人工审查包。

最关键的解释是：框架不会只给出“哪个变体最好”。它会给出“为什么当前方法被选中、它相对 baseline 好在哪里、它在哪些世界失败、哪些 claim 不能说、下一轮应该改什么”。

9. 当前工程边界

当前版本已经完成本地 offline framework 闭环，但仍有边界：

- Synthetic replay 是方法开发基准，不等同于真实湿实验结论。
- Path guard 是 snapshot-based guard，不是操作系统级 sandbox。
- Live literature search 和 real `--use-codex` 需要在具体任务中单独 smoke test。
- H100、MCP、plugin 化暂时没有纳入当前交付。
- anti-HBsAg 项目没有在本轮推进，只修复了框架层和 identifier-safe IO 问题。

三、一句话总结

Design Scientist 现在的定位是：本地长期运行的计算科研系统。它借 AI-Scientist 的科研产物链和树搜索外壳，借 DeepScientist 的 quest/memory/human takeover 思想，但把科学内核替换成数据驱动的多轮 variant design：文献产生方法假设，Codex 或本地节点实现策略，synthetic multi-world replay 做确定性裁判，validation 和 human review 决定是否能进入下一轮真实实验。